

Feature Scaling: Normalization

100 Days of Machine Learning | Day 25 | CampusX

1. What is Normalization?

Normalization is a Feature Scaling technique used to change the values of numerical columns in the dataset so they share a common scale, without distorting differences in the ranges of values. By eliminating the "units" of measurement (like kg vs. cm), the Machine Learning algorithms can process the data purely based on magnitude.

2. Types of Normalization

There are multiple ways to normalize data. The most common ones covered in this session are:

1. **Min-Max Scaling** (Often just called "Normalization")
2. **Mean Normalization**
3. **Max Absolute Scaling**
4. **Robust Scaling**

3. Min-Max Scaling (The Most Common)

When people say "Normalization," they are almost always referring to Min-Max Scaling. It scales all data points so that they fall strictly between **0 and 1**.

$$X_{\text{new}} = (X_i - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

Geometric Intuition:

If you have a 2D scatter plot, Min-Max Scaling picks up your entire data distribution and "compresses" or "squeezes" it perfectly inside a 1x1 unit square (from 0 to 1 on both the X and Y axes) without altering the original shape of the scatter plot.

Scikit-Learn Implementation:

```
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=0)

scaler = MinMaxScaler()
scaler.fit(X_train) # Fit ONLY on training data

X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

4. Other Normalization Techniques

A. Mean Normalization

Centers the data around 0 by using the Mean. The resulting values fall between **-1 and 1**. This is very rarely used because Standardization usually does a better job if centering is required. Scikit-Learn does not have a built-in class for this.

$$X_{\text{new}} = (X_i - X_{\text{mean}}) / (X_{\text{max}} - X_{\text{min}})$$

B. Max Absolute Scaling (MaxAbsScaler)

Divides every value by the absolute maximum value in that column. It is highly recommended when dealing with **Sparse Data** (datasets containing a lot of zeroes).

$$X_{\text{new}} = X_i / |X_{\text{max}}|$$

```
from sklearn.preprocessing import MaxAbsScaler
scaler = MaxAbsScaler()
```

C. Robust Scaling (RobustScaler)

Standard Min-Max Scaling performs terribly if there are outliers, because the `X_max` or `X_min` values get heavily skewed. **Robust Scaling** solves this by relying on the Median and the Interquartile Range (IQR), making it completely **robust to outliers**.

$$X_{\text{new}} = (X_i - X_{\text{median}}) / \text{IQR}$$

```
from sklearn.preprocessing import RobustScaler  
scaler = RobustScaler()
```

5. Standardization vs. Normalization (When to use what?)

A very common confusion for beginners is deciding between Standardization and Normalization.

- **Rule of Thumb 1:** Always ask first: *"Does my algorithm even need scaling?"* (Tree-based algorithms like Random Forest and Decision Trees do NOT need scaling).
- **Rule of Thumb 2:** In most general Machine Learning use-cases, **Standardization (StandardScaler)** yields better results and should be your default choice.
- **When to explicitly use Min-Max Scaling?** Use it when you already know the strict boundaries (Min and Max) of your data beforehand. The most famous example is **Image Processing (CNNs)**, where you know pixel values will strictly range from 0 to 255. Min-Max scaling is perfect for squishing pixel values to a 0-1 range.
- **When to use RobustScaler?** Use it if you know your dataset contains heavy outliers that you cannot remove.
- **When to use MaxAbsScaler?** Use it if you have sparse data (lots of 0s).

Pro Tip: If you are unsure, experiment! Apply all of them, evaluate your model, and see which scaling technique gives you the highest accuracy.